

AI ENGINEERING CODE SUMMIT

FULL REPORT 2025

A select summit of leading coding agent builders and superusers: top AI engineers, engineering leaders (CTOs/VPs), and researchers advancing code generation and AI software engineering.

Abstract: Two days of sessions from experts at the frontier of real world AI engineering revealed a clear architecture: the entire software development life cycle is going through a period of massive transformation, capability beats scaffolding, cleanliness beats cleverness, and human judgment remains the irreducible core.

EXECUTIVE SUMMARY

STATE OF THE FIELD

The AI engineering landscape is transitioning from "pilot purgatory" to production reality at various stages of maturity. Bottom up power-users are redefining what it means to be a software developer while the perennial issues of software engineering - complexity and an "accumulations of whatevers" - have not magically disappeared. The software development lifecycle is fully experiencing AI-future shock.

How much are we actually accomplishing? How much "productivity" is in fact mere reworking the slop that got churned out yesterday? The answer seems to be based on the sophistication of your verification process, and the deeper and more significant your validation process is the closer you get to this tantalizing idea of intelligent agents.

Context management was a recurring theme: text files still reign supreme (now and forever), explicit planning or research phases, either manual coaching or more of a built in framework such as Kiro Specs or Antigravity's Artifacts, and subagents as a general pattern for managing context. And they don't need to be some elaborate memory thing.

The models are so smart now that a lot of the specific tweaks of yester-month to get the harness tuned seems unnecessary, with the Claude Code crew talking about getting "bash-pilled" and Terminus being the one tool needed for these agents to do anything.

Maybe a terminal and a filesystem is all you need.

— Will Schenk

CONTENTS

LEADERSHIP TRACK

1. Trust, Role Changes, and What's Good for Humans is Good for AI
What's good for humans is good for AI - same infrastructure investments benefit both
2. Context, Memory, and The Harness: Where Product Differentiation Really Happens
76% of developers don't trust AI code because they don't trust the context
3. SDLC Evolution: Good Software Engineering Practices, Amplified
AI doesn't eliminate need for good process - it makes good process more valuable
4. The ROI Reality Check: Where AI Actually Delivers (And Where It Falls Flat)
93% of organizations stuck in 'pilot purgatory' - only 7% believe they're at scale
5. Proactive vs Reactive: The UX Evolution
Goal is reducing mental load, not just adding speed
6. The Economics of AI Engineering: New Cost Structures & Compensation Models
AI changes the cost function of engineering - previous cost-per-feature calculations obsolete

ENGINEERING TRACK

1. DO NOT OUTSOURCE THE THINKING: Context Engineering & Human-AI Collaboration
AI can only amplify the thinking you've already done
2. Environments as Universal Abstraction: The New Unit of Everything
A benchmark = environment + starting state + verifier (same as RL environments)
3. Reinforcement Learning for Specialized Models: The Economics of Domain Expertise
1000 examples can yield 10-point improvements with ARFT
4. Skills and Artifacts: The New Building Blocks for Agents
Skills are just folders - procedural knowledge packaged for agents to load dynamically
5. Model Quality Over Scaffolding: Minimalism in Agent Architecture
Use smart models to plan, fast models to execute - deploy the right capability at the right time
6. Data Collection & Quality as the New Bottleneck
Agents are collecting good data but not sharing it - keeping datasets closed slows research

APPENDIX

1. About This Report
2. About the Author

LEADERSHIP TRACK

AI rewards excellence and punishes mediocrity. Trust grows from infrastructure, not promises.

IN THIS SECTION

AI rewards excellence and punishes mediocrity. Trust grows from infrastructure, not promises.

1. Trust, Role Changes, and What's Good for Humans is Good for AI

What's good for humans is good for AI - same infrastructure investments benefit both · Trust grows from predictable behavior, which requires solid infrastructure · Psychological safety is the overwhelming predictor of team success · AI will reshape organizations 100x larger than agile, cloud, CI/CD combined

2. Context, Memory, and The Harness: Where Product Differentiation Really Happens

76% of developers don't trust AI code because they don't trust the context · The harness is the competitive advantage - models commoditize, abstraction layers don't · OpenAI processes dozens of trillions of tokens per week through their harness · Steerability = intelligence + habit - the harness drives steering better than prompts

3. SDLC Evolution: Good Software Engineering Practices, Amplified

AI doesn't eliminate need for good process - it makes good process more valuable · Clean code amplifies AI gains dramatically - environment cleanliness predicts productivity · Code review is the new bottleneck - more PRs, not bugger PRs · Every software developer becomes a code reviewer

4. The ROI Reality Check: Where AI Actually Delivers (And Where It Falls Flat)

93% of organizations stuck in 'pilot purgatory' - only 7% believe they're at scale · AI delivers 35-40% gains on greenfield/low-complexity, but 0-10% on brownfield/high-complexity · Gap between demo and production is 'so broad' in risk-averse environments · Clean code amplifies gains; messy code creates negative feedback loops

5. Proactive vs Reactive: The UX Evolution

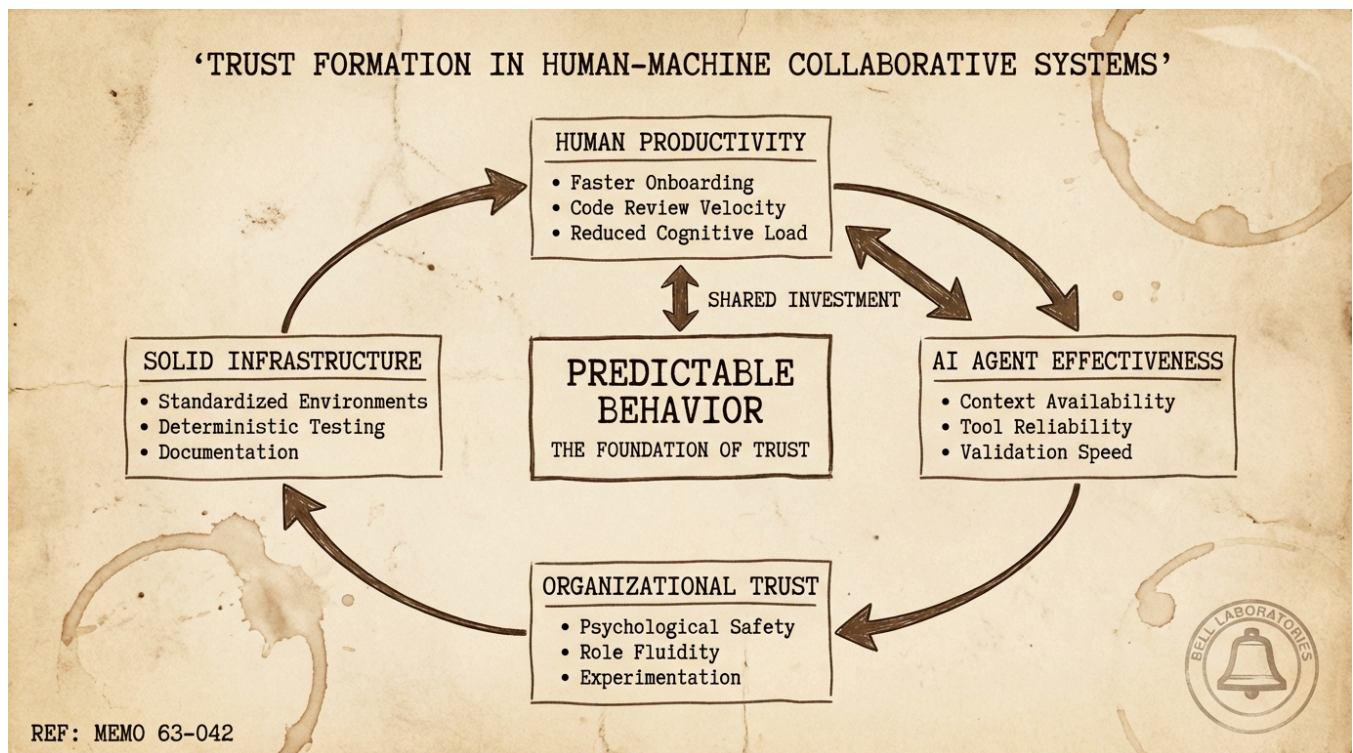
Goal is reducing mental load, not just adding speed · Current tools are 'fundamentally reactive' - forcing constant context-switching · Proactive agents: observation, timely action, personalization, seamless integration · The patterns of how we use an IDE right now might not exist at all next year

6. The Economics of AI Engineering: New Cost Structures & Compensation Models

AI changes the cost function of engineering - previous cost-per-feature calculations obsolete · Tenex pays engineers like salespeople: output-based, uncapped, multiple on track for \$1M+ · Usage-based pricing vs seat-based: one complex query might consume more resources than a thousand simple ones · Five industry trends: LLM-ready data fabric, specialist models, invisible copilots, secure models, usage-based pricing

TRUST, ROLE CHANGES, AND WHAT'S GOOD FOR HUMANS IS GOOD FOR AI

Key Insight: What's good for humans is good for AI - same infrastructure investments benefit both



FEATURED EXPERTS

[Max Kanat-Alexander](#) (Capital One) [web] · [Lisa Orr](#) (Zapier) [web] · [Dan Shipper](#) (Every) [web] · [Gene Kim](#) (IT Revolution) [web] · [Justin Reock](#) (DX) [web]

Gene Kim opened with a striking claim: “AI will reshape technology organizations—100x larger than the impact of agile, cloud, CI/CD, mobile, etc.” The entire economy re-organizes itself around new modes of production. We’re starting to see the case studies.

Our Belief and Observations

- AI will reshape technology organizations — 100x larger than the impact of agile, cloud, CI/CD, mobile, etc.
- These technology breakthroughs reshape organizations... the entire economy re-organizes itself to take advantage of these new modes of production
- We are starting to see amazing case studies hinting at what the future may look like

@RealGeneKim

Kim’s framework for embracing this shift: FAAFO. Faster—obvious. Ambitious —“impossible becomes passive, annoying things become free.” Alone—build more autonomously, smaller teams. Fun—enjoy the work again. Options—more swings at bat. His calculation: \$100 per day in AI tokens generates output comparable to a developer’s annual productivity. When costs collapse that dramatically, every shelved project deserves a second look.

Kim also offered a definition: “Trust is to what degree I can predict how another party will act.” AI trust grows the same way human trust does—through predictable behavior. Stop building special accommodations for agents. Start fixing the foundational problems that have plagued human developers for decades.

Justin Reock from [DX](#) cited Google’s [Project Aristotle](#) research: “the overwhelming predictor [of team success] was psychological safety.” His guidance: “Treat software development as a systems problem, not a people’s problem.” When organizations frame AI adoption as trust-building rather than competency testing, resistance drops.

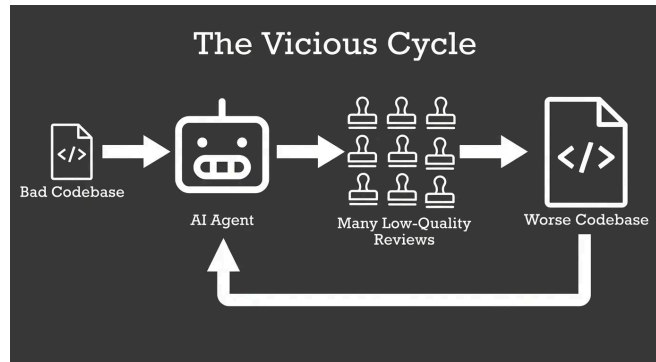
1. 10x difference when you hit 100% AI adoption.
2. A single engineer should be able to build and maintain a complex, production product.
3. Compounding engineering makes each feature easier to build.
4. There are many non-obvious second-order effects once you adopt it.
5. Many people in SF don’t know this yet.

Reock’s predictions for full AI adoption: 10x productivity difference. A single engineer building and maintaining complex production products. Compounding engineering where each feature makes the next easier. “Many people in SF don’t know this yet.”

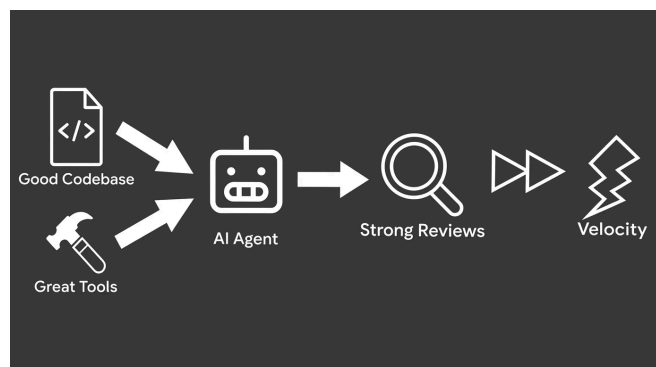
Max Kanat-Alexander, Capital One’s Executive Distinguished Engineer for Developer Experience, put it simply: “What’s good for humans is good for AI.” The same infrastructure investments that make human developers productive—better tooling, clearer documentation, robust testing—are exactly what AI agents need.

Kanat-Alexander’s prescription: standardize environments, improve deterministic validation, refactor for testability, write down external context and intentions. Agents “did not attend your verbal meeting that has no

transcript.” Tribal knowledge kills onboarding—for humans and AI alike.



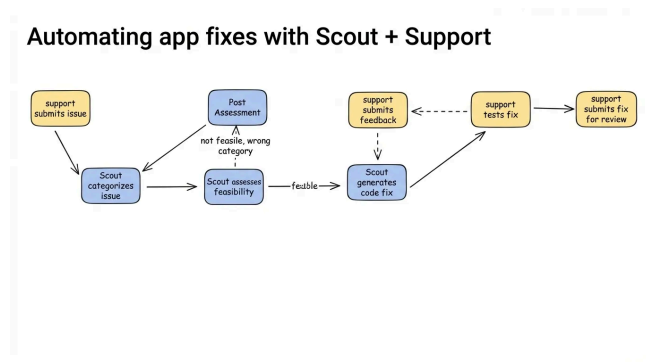
The vicious cycle: bad codebases feed AI agents garbage, producing low-quality reviews that make the codebase worse. The feedback loop accelerates decline.



The virtuous cycle: good codebases and great tools enable AI agents to provide strong reviews, which leads to velocity. Same investment, opposite outcomes.

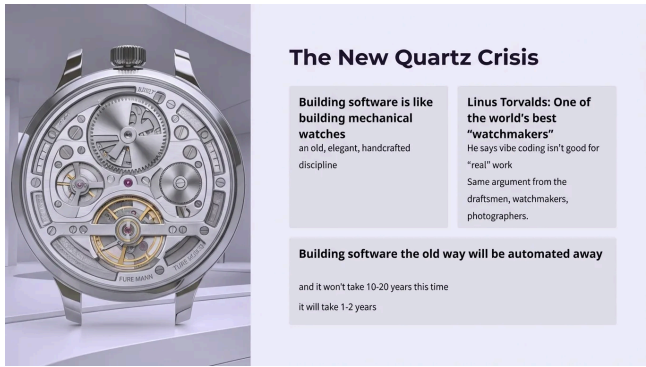
Lisa Orr from [Zapier](#): “Support team members who are part of this experiment are moving into the engineering team.” Not CS graduates finally getting their shot—customer support specialists who understand user pain intimately, now shipping fixes directly. Their

tool Scout helps diagnose issues, generate fixes, and validate solutions with roughly 70% accuracy. Zapier’s support team doubled their velocity, pushing 3-4 fixes per week.



Dan Shipper’s [Every](#)—15 people, six business units, four software products, seven-figure revenue—runs on a radical principle: “Managers can commit code.” Each app is built by one developer working with AI agents in parallel. New hires ship on day one. Shipper calls it “compounding engineering”—each feature makes the next easier rather than adding debt. “AI permits engineers to work with fractured attention.”

The resistance patterns keep repeating. Steve Yegge compared it to the Quartz Crisis that disrupted Swiss watchmaking: “Building software is like building mechanical watches: an old, elegant, handcrafted discipline.” Linus Torvalds says vibe coding isn’t good for “real” work—same argument from draftsmen, watchmakers, photographers. “Building software the old way will be automated away—and it won’t take 10-20 years this time. It will take 1-2 years.”



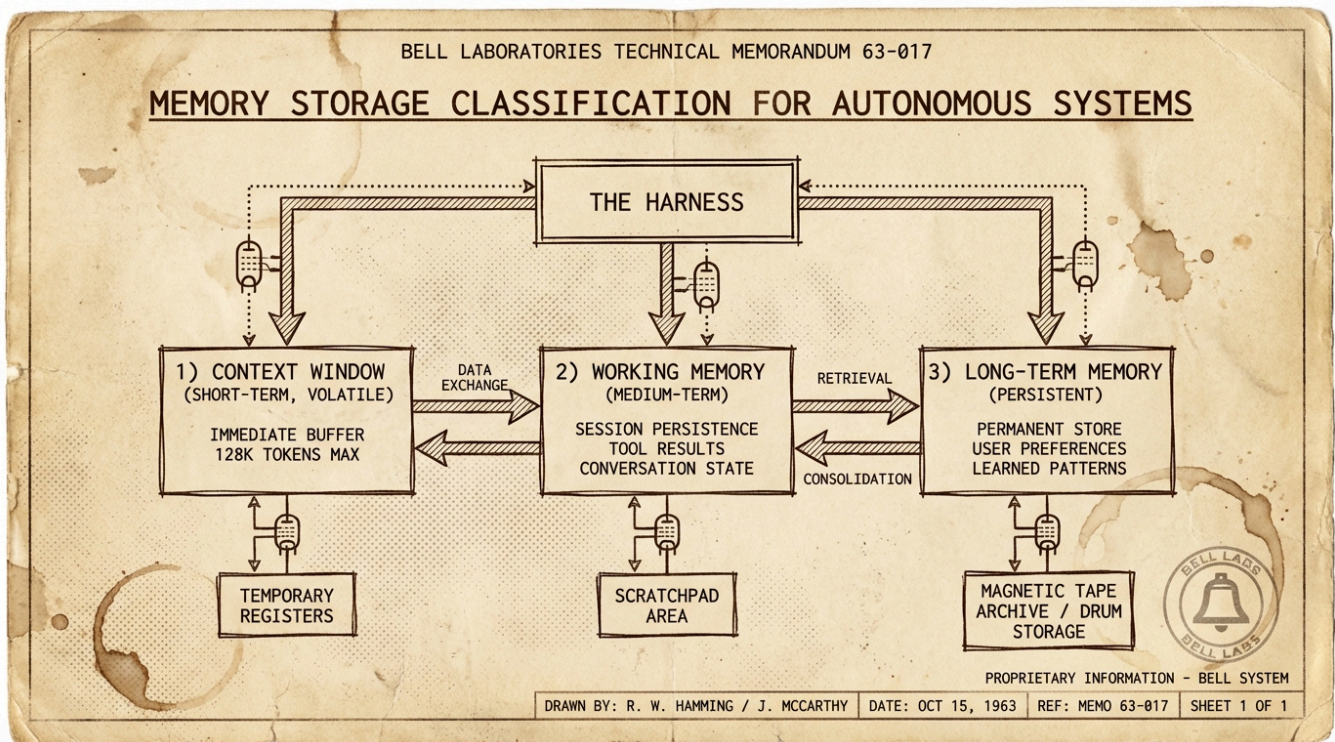
Kim shared war stories: when Capital One and Fidelity pushed senior developers to adopt DevOps, “senior devs didn’t want to do it.” Booking.com elevated developer productivity org-wide. Travelopitia replaced legacy apps in

4-6 weeks with half the team. Now Cisco requires every senior dev to “vibe code an application to production”—same pushback. The dividing line isn’t skill. It’s adaptability.

Trust grows from predictable behavior. The organizations succeeding aren’t deploying fancy models—they’re fixing broken builds, documenting assumptions, creating deterministic tests. Support teams ship code because the system became predictable enough for non-specialists to contribute safely. Managers commit code because the scaffolding supports occasional contributors. Same investments, everyone benefits.

CONTEXT, MEMORY, AND THE HARNESS: WHERE PRODUCT DIFFERENTIATION REALLY HAPPENS

Key Insight: 76% of developers don't trust AI code because they don't trust the context



FEATURED EXPERTS

[Bill Chen](#) (OpenAI) · [Brian Fioca](#) (OpenAI) [[web](#)] · [Michele Catasta](#) (Replit) · [Katelyn Lesse](#) (Anthropic) · [Itamar Friedman](#) (Qodo) [[web](#)] · [Samir Mody](#) (The Browser Company)